ChatGPT

AI-Driven Video Recommendation for Students

Many students struggle when classroom lectures are insufficient or misaligned with the exam syllabus. They often resort to YouTube for alternative explanations, but manually finding relevant and well-structured videos is time-consuming. An **AI-powered learning assistant** can solve this by aggregating a student's available resources (syllabus, notes, playlists, past papers) and recommending an ordered sequence of videos that covers the curriculum. The proposed system takes as input the student's preferred YouTube playlists (for the subject), the upcoming exam syllabus, the teacher's lecture notes (scanned PDFs or even photos of handwritten notes), and past exam papers. It processes these inputs to extract key topics and then selects and orders videos (from any playlists or searches) to cover each topic. If a topic lacks a video, it flags this gap so the student knows where self-study is needed. The AI system also supports on-demand topic search: the student can request videos for a specific topic and receive a curated list. Feedback from the student (for example, correcting whether a video does cover a topic) is used to refine future recommendations. Overall, the goal is to create a **smooth learning path** – students click through videos in order, each building on the previous, so they master the full syllabus efficiently.

System Inputs and Preprocessing

- **YouTube Playlists:** The student provides one or more YouTube playlist links for the subject. These are treated as the primary content pool. The system uses the YouTube Data API (or a transcript library) to download metadata and transcripts of all videos in these playlists. Transcripts give text that can be analyzed for topic content. If multiple playlists cover the same subject, the system merges their videos into a unified content database.

- **Syllabus & Exam Papers:** The syllabus and past exam papers (usually PDF or text) are uploaded. These contain the list of topics and the focus areas. The system parses this text (using NLP tools) to extract **keywords and concepts**. For example, it might use named-entity recognition or keyphrase extraction on the syllabus document to identify the main topics of each chapter. Exam questions can also be analyzed (e.g. via question classification or TF–IDF) to highlight frequently tested concepts.

- **Class Notes (PDF/Handwritten):** The teacher's lecture notes – printed handouts or scanned handwriting – are digitized via OCR (Optical Character Recognition). Modern OCR engines (like Google Vision or Tesseract) can convert printed text to digital text, and specialized Handwritten Text Recognition (HTR) tools can handle neat handwriting. The extracted text is then tokenized and processed to find important terms. For example, each note page can be summarized or keywords extracted. This helps align the videos to the instructor's actual content. In summary, all inputs (syllabus text, exam text, notes) are transformed into a common textual format and tokenized so that their topics can be compared to those in the video transcripts.

By the end of preprocessing, the system has: (a) a set of syllabus topics, (b) a set of exam-focused topics, (c) a set of lecture-note topics, and (d) a collection of transcripts/text from each video in the playlists.

Content Analysis and Topic Matching

The core of the AI system is to **match video content to the needed topics**. To do this, the system builds a topic-based representation of all the content. One approach is to use topic modeling (e.g. Latent Dirichlet Allocation – LDA) or document embeddings on the text from videos and from the input documents. For example, Zhao *et al.* demonstrate using LDA on video transcripts to discover latent topics across the collection [1]. Similarly, we can run LDA or a transformer-based topic extractor on the transcripts and also on the syllabus/notes. Each piece of text (each video transcript, each syllabus section, etc.) becomes a "document" with a distribution over topics. Then matching involves comparing these distributions or embeddings. For example, if Topic A appears prominently in a video's transcript and also in the syllabus or notes, that video is considered relevant for Topic A.

In practice, the system might also use **vector embeddings** (e.g. BERT or sentence-transformers) to compute similarity scores between topic queries and video transcripts. For instance, each topic name or syllabus bullet point can be embedded into a vector, and each video transcript can be embedded; videos with high cosine similarity are deemed on-topic. Topic modeling and embeddings both transform heterogeneous content into comparable representations. Using these techniques, the system identifies which videos (and which time segments, if desired) cover each required topic from the syllabus/notes.

This content matching goes beyond simple keyword search. Because vocabulary can differ between sources (a professor's notes versus a video's phrasing), topic modeling helps capture latent concept connections [1]. By building a "common topic-based representation" as done by Zhao *et al.*, we can robustly link videos to the student's curriculum [2]. In short, NLP and unsupervised learning let the AI detect which video covers which concept.

Recommendation Generation and Sequencing

Once the relevant videos for each topic are identified, the system must **order them into a coherent learning path**. The goal is that the student can watch the recommended videos one after another and cover the entire syllabus without jumping all over. Two key ideas help here:

- **Playlist continuity:** The system tries to keep recommendations from the same playlist (or at least the same channel/instructor) in sequence, if possible. If the student provided multiple playlists, the AI can prioritize videos from one playlist until that section of the syllabus is covered, then switch to another playlist. The rationale is to maintain a consistent teaching style. For example, if Playlist 1 has videos on Topic X1 and X2, and Playlist 2 has another video on X2, it may prefer to show Playlist 1's X2 first, then only use Playlist 2's X2 if a different perspective is needed.

- **Sequential topic transitions:** We also want the sequence of videos to follow a logical progression of concepts. Zhao *et al.* used the structure of course syllabi to mine frequent topic transition patterns and enforce them when re-ranking videos [2]. In our system, we can mimic this by respecting the order of topics in the syllabus or the teacher's notes. If the syllabus lists Topic A before Topic B, the AI will try to recommend videos on A before those on B. Where possible, we also look at transitional coherence: for example, if Video 1 explains part 1 of a concept and Video 2 covers part 2, we put Video 1 then Video 2. If a playlist already has videos in order, we follow that order.

Concretely, the system could treat each topic as a node in a graph and link videos that share topics. It then finds a path that covers all required nodes in order. Techniques like frequent sequence mining can be applied: Zhao's "sequential rule mining" method finds that "Topic Y follows Topic X" often, and uses that to guide suggestions [2]. Similarly, we may look at transcript overlaps: if one video leads naturally into another (detected by content similarity or even by explicit references in speech), those videos are chained together.

If multiple candidate videos cover the same topic, they can be scored/ranked. For ranking, we borrow ideas from fuzzy logic or hybrid filters: one can combine content relevance (the similarity score) with video metadata (duration, view count, recency, ratings). Rishad *et al.* propose a fuzzy ranking that considers creation time, watch time, comments, ratings, etc., to decide which video to show first [3]. We can implement a similar scoring: e.g. prefer videos with higher educational value (longer explanation time, good comment sentiment) once relevance is satisfied. The resulting ranked list is then trimmed to a smooth sequence.

For each syllabus topic, the system picks the top-ranked video (or videos) and appends it to the recommendation list. If a topic has *no* suitable video in the playlists (or on YouTube), the AI flags "No recommended video for Topic X" so the student knows to study it from notes or textbooks. The final output is an **ordered list of video links**, each labeled by topic, which the student can click through sequentially.

Interactive and Feedback Features

Beyond the automated flow, the system has user-interactive features:

- **Manual topic search:** The student can enter a specific topic name or phrase and request videos. The AI will filter its knowledge base for that topic and show a sequence of videos (possibly spanning different playlists) just for that topic. This is essentially the same matching process but scoped to one concept. Again, the system ensures the videos are sequenced logically (perhaps by subtopics or increasing depth).

- **Missing topic alerts:** If a search yields no video (or if syllabus analysis found no match), the interface explicitly says "Video not found for [Topic]". This transparency helps students know when to self-study.

- **Feedback loop:** The student can provide feedback on the recommendations. For instance, if the AI suggested a video for Topic X but the student knows that video is already in Playlist Y at a different time, the student can mark "Video exists in my playlist" or "Video irrelevant". The system uses this feedback to update its data (e.g. marking that topic as covered) or to adjust its matching thresholds. Over time, this trains the system to improve its ordering and selection, similar to how interactive recommender systems refine suggestions from user signals.

Web App Prototype and Workflow

The entire system is envisioned as a **web application**. A prototype could be implemented as a single-page app (e.g. React) with a Python backend (Flask or Django). The workflow is roughly:

1. **Login/Project Setup:** The student creates a project and inputs the course information.
2. **Upload Inputs:** Via the UI, the student pastes one or more YouTube playlist URLs, uploads syllabus PDFs or text, uploads scanned notes/past papers (images or PDF).
3. **Backend Processing:** Upon submission, the server runs OCR on images, parses PDFs, fetches YouTube transcripts (using an API or library), and then runs the analysis pipeline (NLP, matching, sequencing).
4. **Recommendation Display:** The frontend shows a list/table of video titles and links, ordered by the system. Each entry shows which topic it covers. Missing topics are clearly marked. There is a separate "Search by topic" box for on-demand queries.
5. **Feedback Input:** Next to each recommendation, the student can click "Correct", "Wrong topic", or "Already in my playlist" etc. This feedback is sent back to refine the model.

This pipeline is similar to other web-based educational recommender concepts. For example, Jabar *et al.* describe a system where a student query triggers a YouTube search and analysis, after which ranked educational videos are recommended via a web app [4]. Our system generalizes this by using richer inputs (notes, syllabus) instead of just a text query.

Technical Details

Some key implementation details:

- **OCR & Text Extraction:** Use Tesseract or Google Vision for printed text, and specialized HTR tools for neat handwriting. After OCR, apply cleanup (spell-check, segmentation into paragraphs or bullet points).
- **NLP & Topic Extraction:** Use Python libraries (spaCy, NLTK, Scikit-learn) or transformer models (BERT) to extract keywords and semantic embeddings. LDA (e.g. via Gensim) can model topics if the text corpus (videos+notes+syllabus) is large enough. Zhao *et al.* note that LDA effectively models educational content topics across videos [1].
- **Video Transcripts:** Acquire transcripts using the YouTube API (if captions exist) or third-party tools. If videos lack transcripts, one could use an automated speech recognition (ASR) service to generate them. The transcripts are then processed identically to text from PDFs.
- **Similarity/Matching:** For each syllabus topic phrase or note keyword, compute its vector and find the nearest videos. Alternatively, simply compute TF–IDF cosine similarity between each topic "query document" and each video transcript. Set a threshold to consider "relevant".
- **Sequencing Algorithm:** Sort matched videos first by an inferred topic order (e.g. syllabus order), then refine by metadata ranking. One can post-process to ensure minimal topic jumps. Zhao's method of mining frequent transitions from curricula can be adapted: e.g. if "derivatives" often follows "limits", ensure that order if both topics appear [2].
- **Ranking/Fuzzy Logic:** Implement fuzzy scoring as suggested by Rishad *et al.* [3]. Weight factors like relevance, video length (too short might be insufficient), view count (as proxy for quality), and user ratings.

- **Front-end UI:** Simple, with sections for each topic. Each video link could display a thumbnail, title, and short summary (maybe the first sentence of transcript or description). Missing-topic items could have a “warning” icon. Provide drag-and-drop reordering or checkboxes for feedback.

Since we only need to provide YouTube links (not host any videos), licensing is simple: all recommended content is publicly available. Most educational videos are free to watch. The AI system essentially acts as a personalized index.

Benefits and Impact

This tool would greatly improve study productivity. Instead of manually trawling YouTube or lecture slides, the student gets a **customized study path**. They see exactly what to watch in what order to cover the syllabus. This avoids redundant topics and fills knowledge gaps. By aligning videos to the teacher’s own notes and exam focus, the learning becomes highly relevant. Students from any department or background could use the system for any course – the process is generic. The main outcome is stress reduction before exams, as learners can quickly grasp difficult concepts through handpicked videos that flow logically.

Existing research shows that such syllabus-driven video recommendations can enhance learning. For instance, Rishad *et al.* describe the *Acharya* platform, a syllabus-centric video repository where users tag videos to topics; they emphasize ranking videos by relevance and quality (using comment sentiment, watch time, etc.) in a fuzzy logic framework [3]. Our idea similarly tailors video selection to syllabus topics. Zhao *et al.* also find that using syllabus structure to order video topics leads to better educational recommendations [2]. By combining these insights, our system aims to provide a smoother transition between videos and full coverage of required content.

In summary, this AI-powered recommender would be a **web-based prototype** that students can interact with. It automates the hard part of finding and sequencing videos, leaving students to simply watch the recommended list. Over time, the model can improve via feedback and possibly expand beyond YouTube to include other open lecture sources. But even in its prototype form, linking to YouTube videos suffices, since YouTube is open-access for students. The result is a practical study assistant that transforms a student’s favorite playlists and notes into a full exam study plan, all ordered by AI to maximize learning efficiency.

References: Existing methods for educational video recommendation use topic modeling and syllabus ordering to improve recommendations [2] [1]. Fuzzy logic and user-context can rank videos by relevance and quality [3]. A typical web-based recommender pipeline fetches YouTube data and applies such analytics to output curated video lists [4]. These ideas inform our design to ensure comprehensive and coherent video recommendations.

---

[1] [2] jeffjianzhao.com
https://www.jeffjianzhao.com/papers/seqsense.pdf

[3] Application of fuzzy logic in video recommendation system for syllabus driven E-learning platform | Request PDF
https://www.researchgate.net/publication/350418191_Application_of_fuzzy_logic_in_video_recommendation_system_for_syllabus_driven_E-learning_platform

4 Educational Video Recommender System
https://www.ijiet.org/vol14/IJIET-V14N3-2058.pdf